

Conteo de frecuencia de palabras

Guia para entender el programa

Version secuencial y version paralela con POSIX threads

Medicion de tiempo, speedup y efficiency

Para que sirve este documento. Explica, de principio a fin, que hace el programa, como se paralelizo, por que se midio el tiempo donde se midio, y que significan los numeros que obtuvimos. Al final hay una seccion de preguntas frecuentes con las respuestas listas, por si las preguntan al exponer.

Hoja de Trabajo 1 — Computacion Paralela y Distribuida

11 de agosto de 2026

1. Que hace el programa

El programa lee un archivo de texto y cuenta **cuantas veces aparece cada palabra**. El resultado es una tabla ordenada alfabeticamente con cada palabra distinta y su frecuencia.

```
$ ./conteo_secuencial archivo.txt
```

Palabra	Frecuencia
algoritmo	12
computacion	7
de	214
paralela	19
...	
Total de palabras diferentes: 904	

Antes de contar, cada palabra se **normaliza**: se pasa a minusculas y se descarta todo lo que no sea letra o numero. Asi Casa, casa y casa, cuentan como la misma palabra.

Un caracter forma parte de una palabra si cumple:

```
static int es_caracter_de_palabra(unsigned char caracter)
{
    /* Los bytes >= 128 permiten conservar letras acentuadas en UTF-8. */
    return isalnum(caracter) || caracter >= 128;
}
```

Cualquier otra cosa —espacio, coma, punto, salto de linea— marca el **final** de la palabra actual. Esa es toda la regla de separacion.

La estructura de datos

Las frecuencias se guardan en un arreglo dinamico de pares (*palabra, frecuencia*) que crece al doble cuando se llena:

```
typedef struct {
    char    *palabra;      /* copia propia de la palabra */
    size_t  frecuencia;    /* cuantas veces aparecio      */
} EntradaFrecuencia;

typedef struct {
    EntradaFrecuencia *entradas;
    size_t cantidad;      /* cuantas usadas              */
    size_t capacidad;      /* cuantas caben               */
} TablaFrecuencias;
```

Ojo con esto, es importante para entender los resultados. Para insertar una palabra, `agregar_palabra()` recorre la tabla **de principio a fin** comparando con `strcmp`. Con 904 palabras distintas, cada insercion cuesta hasta 904 comparaciones: el algoritmo es **O(n²)**. Es lento, pero es justamente lo que hace que valga la pena paralelizarlo, porque hay bastante computo real que repartir.

2. La version secuencial

Un solo flujo de ejecucion recorre el texto de principio a fin, caracter por caracter, armando cada palabra y metiendola en la tabla:

```
for (i = inicio; i < fin; i++) {
    unsigned char actual = (unsigned char)texto[i];

    if (es_caracter_de_palabra(actual)) {
        palabra[longitud++] = tolower(actual); /* sigue la palabra */
    } else if (longitud > 0) {
        palabra[longitud] = '\0'; /* termino: guardarla */
        agregar_palabra(tabla, palabra, 1);
        longitud = 0;
    }
}
if (longitud > 0) { /* por si el texto no termina en separador */
    palabra[longitud] = '\0';
    agregar_palabra(tabla, palabra, 1);
}
```

Ese ultimo if no es un detalle menor: si el archivo termina justo con una letra y no con un salto de linea, sin el se perderia la ultima palabra.

Que le cambiamos respecto al Corto 02

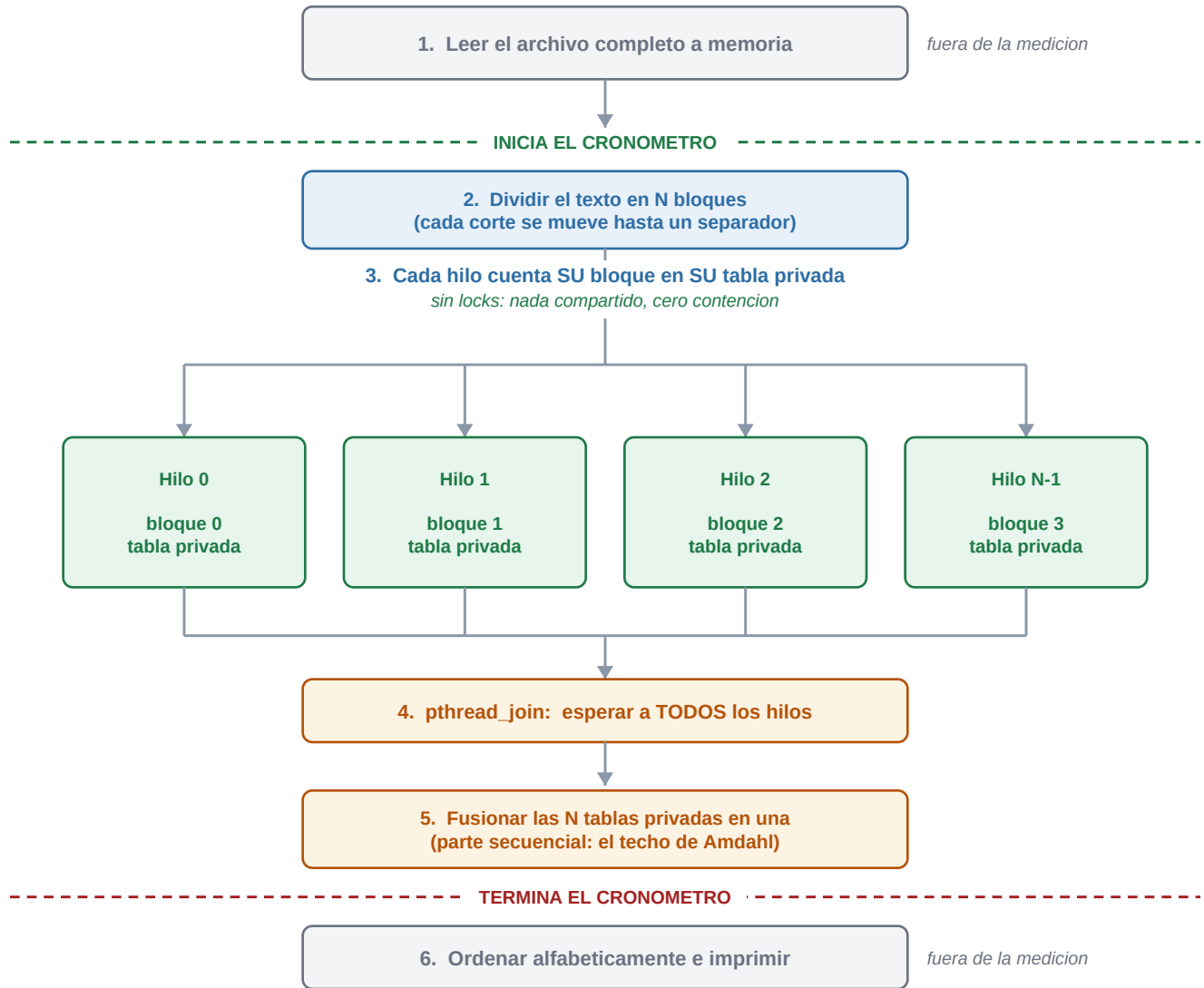
La logica de conteo es la misma que ya teniamos. Cambiaron tres cosas, y las tres existen para poder medir el tiempo de forma honesta:

- **Se separo la lectura del conteo.** Antes se leia con `fgetc()` caracter por caracter *mientras* se contaba. Ahora el archivo se carga completo a memoria **antes** de arrancar el cronometro. Si no hicieramos esto, estaríamos midiendo el disco en vez del procesador.
- **El conteo se aislo en una funcion.** `contar_palabras(texto, inicio, fin, tabla)` recibe un rango. La secuencial la llama una vez sobre todo el texto; en la paralela, cada hilo la llama sobre su pedazo. **Es literalmente la misma funcion en las dos versiones**, y por eso la comparacion de tiempos es valida.
- `agregar_palabra` **recibe un parametro** veces. La secuencial siempre pasa 1. Sirve para la paralela, donde al fusionar hay que sumar de golpe "esta palabra aparecio 7 veces en el bloque del hilo 2".

Regla de oro al comparar versiones: si el cronometro no cubre exactamente el mismo trabajo en ambas, el speedup no significa nada. Por eso la lectura del archivo, el ordenamiento y la impresion quedan **fuera** de la medicion en las dos.

3. La version paralela

La idea es simple: si el texto se parte en N pedazos, N hilos pueden contar al mismo tiempo, cada uno en su pedazo. Luego se juntan los resultados. El diagrama muestra el flujo completo.



Flujo de la version paralela. Solo la zona entre las lineas punteadas se mide.

Etapa 2: dividir el texto sin partir palabras

Este es el **detalle mas facil de arruinar**. Si cortamos el texto en pedazos iguales sin cuidado, un corte puede caer en medio de una palabra:

```
...el algoritmo para | lelo cuenta...
                    ^
                corte a la mitad de "paralelo"

Hilo 0 contaria "para"  (que no existe)
Hilo 1 contaria "lelo"  (que tampoco)
y "paralelo" desapareceria del conteo.
```

La solucion: calcular el corte por division y luego **avanzarlo hacia adelante** hasta encontrar un separador.

```
corte = tamano * (i + 1) / num_hilos;

/* Avanzar hasta el siguiente separador para no partir palabras. */
while (corte < tamano && es_caracter_de_palabra((unsigned char)texto[corte])) {
    corte++;
}
```

Como consecuencia los bloques no quedan exactamente del mismo tamano —uno puede tener unos bytes de mas—, pero cada palabra queda completa dentro de un solo bloque. Ese pequeno desbalance es una de las fuentes de overhead.

Etapa 3: por que cada hilo tiene su PROPIA tabla

Esta es la decision de diseno mas importante del programa. Cada hilo escribe en una tabla que solo el toca:

```
typedef struct {
    const char *texto;
    size_t inicio, fin;      /* su bloque: [inicio, fin) */
    TablaFrecuencias tabla; /* SU tabla, nadie mas la toca */
    int ok;
} TareaHilo;
```

La alternativa obvia —una sola tabla compartida protegida con un mutex— **habria sido un desastre**. Como practicamente toda la ejecucion consiste en insertar palabras en la tabla, cada hilo pasaria casi todo su tiempo esperando el candado. El programa quedaria serializado en la practica y, sumando el costo de pedir y soltar el mutex millones de veces, muy probablemente correria **mas lento que la version secuencial**.

Es la leccion central de la HDT. Paralelizar no es "repartir y ya": es repartir *de forma que los hilos no se estorben*. Al darle a cada hilo datos privados, la fase paralela no tiene ni un solo lock. Todo el costo de coordinacion se concentra en un unico punto: la fusion final.

Etapas 4 y 5: sincronizar y fusionar

`pthread_join` bloquea hasta que el hilo termina. Al hacerlo con los N, el programa no avanza hasta que **todos** acabaron:

```
for (i = 0; i < num_hilos; i++) {
    pthread_join(hilos[i], NULL);    /* esperar a que todos terminen */
}
```

```

}

/* Reduccion: volcar las tablas 1..N-1 sobre la del hilo 0. */
for (i = 1; i < num_hilos; i++) {
    for (j = 0; j < tareas[i].tabla.cantidad; j++) {
        agregar_palabra(&tareas[0].tabla,
                        tareas[i].tabla.entradas[j].palabra,
                        tareas[i].tabla.entradas[j].frecuencia); /* suma de golpe */
    }
}
}

```

Aqui esta el cuello de botella. La fusion la hace **un solo hilo**, mientras los demas ya terminaron y estan ociosos. Es la porcion secuencial del programa, y encima **crece con N**: con 2 hilos hay 1 tabla que fusionar, con 4 hay 3. Esto es exactamente lo que la **ley de Amdahl** describe, y explica por que el speedup nunca llega a ser 4x con 4 hilos.

4. Como se mide el tiempo

Se usa `clock_gettime` con `CLOCK_MONOTONIC`, que tiene resolucion de nanosegundos y —a diferencia del reloj de pared— **nunca retrocede** aunque el sistema ajuste la hora:

```

struct timespec inicio, fin;

clock_gettime(CLOCK_MONOTONIC, &inicio); /* ---- ARRANCA ---- */

    contar_palabras(texto, 0, tamano, &tabla); /* el trabajo real */

clock_gettime(CLOCK_MONOTONIC, &fin); /* ---- PARA ---- */

double tiempo = (fin.tv_sec - inicio.tv_sec)
                + (fin.tv_nsec - inicio.tv_nsec) / 1e9;

```

Que queda dentro y que queda fuera:

	Version secuencial	Version paralela
FUERA (antes)	Leer el archivo a memoria	Leer el archivo a memoria
DENTRO	Recorrer el texto y llenar la tabla	Dividir + crear hilos + contar + join + fusionar
FUERA (despues)	Ordenar e imprimir	Ordenar e imprimir

Fijate que **todo el overhead del paralelismo esta adentro** de la medicion: el reparto, la creacion de hilos, la espera y la fusion. No hicimos trampa midiendo solo la parte bonita. Si alguien pregunta "y el costo de crear los hilos?", la respuesta es que ya esta contado.

Por que 5 ejecuciones y no una

Los tiempos varian bastante entre corridas. Estas son las 5 mediciones secuenciales reales sobre el archivo de 24 KB:

```
#1 0.004635 s      <- la mas lenta
#2 0.004826 s
#3 0.003230 s
#4 0.002752 s
#5 0.002447 s      <- la mas rapida (casi 2x mas rapida que la #2)
```

Casi el doble de diferencia entre la mas lenta y la mas rapida, **con el mismo codigo y el mismo archivo**. Las causas: otros procesos compitiendo por el CPU, el planificador del sistema, la frecuencia del procesador que sube y baja, y el efecto de calentamiento de cache (las primeras corridas son mas lentas porque los datos todavia no estan en cache). Por eso se promedian 5 corridas: una sola medicion no significa nada.

5. Speedup y efficiency

Speedup responde: ¿cuántas veces más rápido corre la versión paralela?

$$\text{Speedup} = \frac{\text{tiempo secuencial}}{\text{tiempo paralelo}} \quad S = 2 \rightarrow \text{el doble de rápido}$$

Efficiency responde: ¿qué tan bien se aprovecha cada hilo? Es el speedup repartido entre los hilos usados.

$$\text{Efficiency} = \frac{\text{Speedup}}{p \text{ hilos}} \quad \begin{array}{ll} E = 1.0 \text{ (100 \%)} & \rightarrow \text{aprovechamiento perfecto} \\ E = 0.5 \text{ (50 \%)} & \rightarrow \text{se desperdicio la mitad} \end{array}$$

No son lo mismo, y confundirlos es el error típico. El speedup mide *velocidad*; la efficiency mide *aprovechamiento*. Una configuración puede ser la más rápida y a la vez la que peor usa los recursos — y eso es exactamente lo que nos paso.

Un ejemplo con nuestros números

Secuencial : 0.003578 s Paralela 4 hilos : 0.001745 s

$$\begin{array}{ll} \text{Speedup} &= 0.003578 / 0.001745 = 2.05 \rightarrow 2.05 \text{ veces más rápido} \\ \text{Efficiency} &= 2.05 / 4 = 0.51 \rightarrow \text{solo 51 \% de aprovechamiento} \end{array}$$

Interpretación: usamos 4 hilos pero solo obtuvimos el rendimiento de 2.05. Casi la mitad de la capacidad se fue en coordinación.

Resultados obtenidos

Equipo de prueba: Apple M5, 10 núcleos (4 de rendimiento + 6 de eficiencia). Promedio de 5 corridas.

Archivo	Configuración	Tiempo prom.	Speedup	Efficiency
archivo.txt 24 KB	Secuencial	0.003578 s	1.00	100 %
	2 hilos	0.002188 s	1.64	81.8 %
	4 hilos	0.001745 s	2.05	51.3 %
archivo_grande.txt 958 KB	Secuencial	0.071592 s	1.00	100 %
	2 hilos	0.042284 s	1.69	84.7 %
	4 hilos	0.024765 s	2.89	72.3 %

6. Que significan los resultados

Hallazgo 1: mas rapido no es mas eficiente

Con 4 hilos obtuvimos **el mayor speedup** (2.05x), pero con 2 hilos obtuvimos **la mayor eficiencia** (81.8 % contra 51.3 %). Son configuraciones ganadoras distintas segun lo que se quiera optimizar: si importa terminar rapido, 4 hilos; si importa no desperdiciar recursos, 2 hilos.

Hallazgo 2: el speedup crece cada vez menos

Al pasar de 2 a 4 hilos duplicamos los recursos, pero el tiempo solo bajo de 0.002188 s a 0.001745 s: un **20 % de mejora a cambio del doble de hilos**. El segundo par de hilos apporto una fraccion minima de lo que apporto el primero.

Hilos	Speedup ideal	Obtenido (24 KB)	Obtenido (958 KB)
2	2.00	1.64 (82 % del ideal)	1.69 (85 % del ideal)
4	4.00	2.05 (51 % del ideal)	2.89 (72 % del ideal)

La brecha contra el ideal **se ensancha** conforme se agregan hilos. Eso es la ley de Amdahl en accion: la fusion de tablas es secuencial y le pone un techo al speedup, sin importar cuantos hilos se agreguen.

Hallazgo 3: el tamano del problema lo cambia todo

Este es el resultado mas interesante del trabajo. **El mismo codigo, sin cambiar una sola linea**, con 4 hilos:

```
archivo.txt          (24 KB)  ->  efficiency 51.3 %
archivo_grande.txt   (958 KB) ->  efficiency 72.3 %
```

La diferencia no esta en el paralelismo sino en **cuanto trabajo util habia para repartir**. El costo de crear hilos y fusionar tablas es practicamente fijo: si el trabajo real es pequeno, ese costo se lo come todo; si es grande, se diluye. Un archivo de 24 KB simplemente **no es lo bastante grande** para justificar 4 hilos.

Hallazgo 4: mas hilos no siempre es mejor

No es solo que el promedio empeore. En corridas individuales sobre el archivo pequeno, **4 hilos fue mas lento que 2**:

```
Corrida #5:  2 hilos = 0.001376 s    4 hilos = 0.001426 s    <- mas lento
Corrida #4:  2 hilos = 0.001510 s    4 hilos = 0.001490 s    <- empate
```

Agregar hilos ahi no apporto absolutamente nada. Nuestro procesador tiene 10 nucleos, pero con este archivo ya a los 4 hilos la eficiencia estaba por debajo del 52 %: usar 10 hilos sobre 24 KB de texto seria **contraproducente**, porque crear y fusionar 10 tablas costaria mas que el conteo mismo.

7. De donde sale el overhead

Todo lo que la version paralela hace y la secuencial no. Ordenado por impacto:

Fuente	Por que cuesta
Fusion de las tablas (la principal)	Es 100 % secuencial y crece con N. Con 4 hilos hay 3 tablas de hasta 904 palabras que fusionar, y cada insercion hace busqueda lineal. Es el techo de Amdahl.
Crear y destruir hilos	Cada pthread_create cuesta decenas de microsegundos. Sobre un trabajo total de 3.5 ms, 4 hilos son un costo fijo nada despreciable.
La espera del join	El tiempo lo marca el hilo mas lento , no el promedio. Los demas quedan ociosos esperando.
Desbalance de carga	Los bloques se reparten por bytes, no por palabras. Un bloque con palabras largas tiene menos palabras que contar que uno con palabras cortas.
Nucleos heterogeneos	El M5 tiene nucleos rapidos (P) y lentos (E). Si un hilo cae en un nucleo E, tarda mas y todos lo esperan en el join.
Memoria duplicada	Con 4 hilos existen 4 tablas simultaneas: 4x la memoria y 4x las llamadas a malloc.
Contencion en malloc	El asignador del sistema usa locks internos. Varios hilos pidiendo memoria a la vez se serializan parcialmente sin que el codigo lo note.

8. Como correrlo

```
make                # compila las dos versiones
make verificar      # comprueba que la paralela da lo mismo que la secuencial
make benchmark       # 5 corridas por configuracion + speedup y efficiency
make clean           # borra los ejecutables

# Con otro archivo o mas repeticiones:
make benchmark ARCHIVO=archivo_grande.txt REPS=10
```

Y a mano, si hace falta:

```
gcc -O2 -Wall        -o conteo_secuencial conteo_frecuencia_secuencial.c
gcc -O2 -Wall -pthread -o conteo_paralelo  conteo_frecuencia_paralelo.c

./conteo_secuencial archivo.txt           # tabla completa + tiempo
./conteo_paralelo  archivo.txt 4           # con 4 hilos
./conteo_paralelo  archivo.txt 4 --silencioso # solo el tiempo
```

La version paralela necesita -pthread y la secuencial no. Si se te olvida, el error es un undefined symbol: _pthread_create al enlazar. Por eso existe el Makefile.

Verificar que ambas versiones dan lo mismo

Un speedup solo vale si las dos versiones calculan lo mismo. `verificar.sh` lo comprueba automaticamente y devuelve codigo de salida 0 o 1:

- Compara la salida completa contra la secuencial con **1, 2, 3, 4, 5, 7, 8, 16 y 32 hilos**. Se incluyen impares y numeros que no dividen el tamaño, para forzar cortes en posiciones raras. Identica en las nueve.
- Prueba **7 casos borde** —archivo vacio, una sola palabra, solo espacios, dos palabras de 3 bytes, palabras repetidas, mayusculas y minusculas, y texto sin salto de linea final— cada uno con 1, 2, 4 y 8 hilos. Los de 3 y 4 bytes son los mas exigentes: con 8 hilos varios reciben bloques vacios.

9. Preguntas frecuentes

Con las respuestas listas, por si las preguntan al exponer.

¿Por que usaron pthreads y no OpenMP?

Porque el enunciado no exige ninguna libreria en particular: pide "al menos dos procesos o hilos" y permite "C, C++ u otro lenguaje". pthreads es parte del estandar POSIX y compila sin instalar nada, mientras que OpenMP en macOS requiere instalar libomp aparte. El analisis seria identico: OpenMP con tablas privadas por hilo hace exactamente lo mismo por debajo.

¿Por que el speedup con 4 hilos no fue 4x?

Por la ley de Amdahl. La fusion de las tablas es secuencial —la hace un solo hilo mientras los demas esperan— y ademas crece con el numero de hilos. A eso se suma el costo de crear los hilos y el desbalance de carga. Ninguna parte secuencial se acelera por agregar hilos, y esa parte le pone un techo al speedup.

Si tienen 10 nucleos, ¿por que no usaron 10 hilos?

Porque no habria ayudado, y con el archivo pequeno habria empeorado. Ya con 4 hilos la eficiencia cayo al 51 %. Con 10 hilos habria que crear y fusionar 10 tablas sobre solo 24 KB de texto: el costo de coordinacion superaria el ahorro del conteo. El numero optimo de hilos depende de cuanto trabajo hay, no de cuantos nucleos tiene la maquina.

¿Por que cada hilo tiene su propia tabla en vez de compartir una?

Porque compartirla exigiria un mutex, y como casi todo el programa consiste en insertar en la tabla, los hilos pasarian el tiempo esperando el candado. Quedaria serializado y probablemente mas lento que la secuencial. Con tablas privadas la fase paralela no tiene ni un lock; el costo se paga una sola vez, al fusionar.

¿Como evitan partir una palabra entre dos hilos?

Cada corte se calcula por division y luego se **avanza hacia adelante** hasta encontrar un separador. Los bloques quedan de tamanos ligeramente distintos, pero cada palabra queda completa dentro de un solo bloque.

¿Como saben que la paralela da el resultado correcto?

Lo verificamos automaticamente con `verificar.sh`: la salida completa —904 palabras con sus frecuencias— es identica a la secuencial con 1, 2, 3, 4, 5, 7, 8, 16 y 32 hilos, mas 7 casos borde probados con 1, 2, 4 y 8 hilos cada uno.

¿Por que promediaron 5 corridas en vez de tomar una sola?

Porque los tiempos varian mucho por causas externas al programa: otros procesos, el planificador, la frecuencia del CPU y el calentamiento de cache. En nuestras mediciones la corrida secuencial mas lenta fue casi el doble que la mas rapida, con el mismo codigo y el mismo archivo. Una sola medicion no seria confiable.

¿Por que la eficiencia baja al agregar hilos?

Porque el overhead crece con el numero de hilos (mas hilos que crear, mas tablas que fusionar) mientras el trabajo util por hilo se reduce. La relacion trabajo util / overhead empeora en cada paso.

¿Por que midieron tambien con un archivo grande?

Para aislar el efecto del tamaño del problema. Con 24 KB los tiempos son de milisegundos y el overhead pesa muchísimo. Al repetir el mismo texto 40 veces y medir de nuevo, la eficiencia con 4 hilos subió de 51 % a 72 % **sin cambiar el código**. Es la evidencia más clara de que el beneficio del paralelismo depende de que haya trabajo suficiente.

¿La medicion incluye el tiempo de leer el archivo?

No, a propósito. Leer del disco no es trabajo de cómputo y depende del cache del sistema operativo; incluirlo distorsionaría la comparación. El archivo se carga completo a memoria **antes** de arrancar el cronómetro en **las dos** versiones. Lo que sí está incluido es todo el overhead del paralelismo.

En una frase. Repartir el texto entre varios hilos sí acelera el conteo, pero cada hilo agregado rinde menos que el anterior, porque la fusión final es secuencial y el costo de coordinación crece. Cuanto más grande el archivo, más vale la pena — con 24 KB, 4 hilos aprovechan la mitad de su capacidad; con 958 KB, casi tres cuartas partes.